Hydrologic Response Unit (HRU) Discretization Walkthrough

In this walkthrough, we demonstrate a workflow for discretizing Hydrologic Response Units (HRUs) for Raven hydrologic modeling in a small catchment (in the Waterloo region). Specifically, we use five core geospatial layers/attributes commonly required in Raven modeling for this demo: subbasin, lake, land cover, vegetation, and soil profile. After constructing HRUs at the native spatial resolution of each dataset, we further simplify the HRU layer by removing sliver HRUs to balance spatial detail (information content) with model efficiency (computational cost). We then compute key geographic and topographic attributes for the final HRUs, such as centroids, areas, and mean elevation, slope, and aspect, which are widely used in hydrologic modeling and parameterization. Finally, we demonstrate how to translate the HRU attributes into a Raven-compatible HRU format (RVH file) to support semi-distributed hydrologic modeling.

This walkthrough is organized into six main sections. Section 1 introduces the study catchment and the geospatial datasets used in the lab. Section 2 summarizes the required software and tools (ArcGIS Pro and a text editor). Section 3 provides a high-level overview of the HRU workflow, from HRU discretization to simplification, attribute calculation, and Raven input preparation. Section 4 presents detailed step-by-step instructions in ArcGIS Pro to implement the workflow and generate the final HRU layer. Section 5 explains how to translate the GIS-based HRU output into Raven-required RVH format for modeling. Section 6 concludes with several discussion questions to reflect on HRU simplification choices, tool limitations, and how data quality affects hydrologic modeling results. A detailed, step-by-step guide for building a semi-automated HRU simplification workflow in ArcGIS is provided in the Appendix.

If you are using this walkthrough for the **ENVE 383 GIS lab**, the HRU simplification procedures in Section 4.4 are *optional*. Students are only required to complete the HRU creation workflow (Section 4.3), then you may skip Section 4.4 and proceed directly to Section 4.5 for the remaining exercises.

The full lab materials are available for download on LEARN or here.

If you encounter any issues not covered in this walkthrough, please contact Hongren Shen (hongren.shen@uwaterloo.ca) or Amir Al-Bayati (aalbayati@uwaterloo.ca) for assistance.

Contents

1. Catchment and Data Sources **3**
2. Software and Tools **4**
3. Workflow Overview: HRU Discretization and Simplification **5**
3.1 HRU discretization (HRU_raw) 5
3.2 HRU simplification (HRU_final) 6
3.3 HRU attribute calculation 7
3.4 HRU file preparation for Raven modeling 7
4. Implementation Details (ArcGIS Pro) **7**
4.1 Project setup and geoprocessing environments 7
4.2 Land cover reclassification and vegetation fields 8
4.3 HRU overlay operations 10
4.3.1 Union (land cover, vegetation, and soil profile) 10
4.3.2 Intersect (HRU_union and subbasin polygons) 11
4.3.3 Erase (Lake override) 12
4.3.4 Merge and Dissolve (HRU_raw) 13
4.4 HRU simplification (Sliver elimination) 16
4.4.1 Compute HRU_area and identify slivers 16
4.4.2 Simplify HRU subbasin by subbasin 16
4.4.3 Add HRU_ID (unique HRU identifier) 21
4.4.4 Join subbasin attributes to HRU layer 21
4.5 HRU attributes calculation 22
4.6 HRU QAQC 22
4.7 HRU export (Shapefile and GeoJSON) 23
5. Raven HRU File Preparation (RVH) **24**
6. Discussions **26**
Appendix A: Building Semi-automated HRU Simplification Tool (HRU_simplify) **28**

1. Catchment and Data Sources

Our study area is the catchment draining to the Water Survey of Canada (WSC) streamflow gauging station 02GA024 (LAUREL CREEK AT WATERLOO) (Figure 1). The total drainage area is 58 km^2. Land cover and land use are dominated by urban areas in the east and agricultural lands in the west, with additional contributions from open water, forest, grassland, and wetlands. Two small lakes are located within the catchment: Laurel Creek Reservoir (area = 0.63 km^2) and Columbia Lake (area = 0.17 km^2). Several datasets are required in our HRU discretization as follows.

(1) River routing network (vector): OLRRP v2

We use the Ontario Lake and River Routing Product version 2 (OLRRP v2**)** river routing network for the 02GA024 catchment. The routing network includes several vector layers: subbasin polygons**,** river polylines**,** lake polygons**,** points of interest (POI), and a basin outline polygon**.** In the original OLRRP v2 dataset, subbasins are provided at relatively high spatial resolution (average subbasin area of 4.3 km^2). For this lab exercise, we use a **simplified network** to keep the workflow manageable and to limit the number of HRU polygons generated during overlay and simplification steps. The provided river network therefore contains **six subbasins**, with an average subbasin area of 9.7 km^2. More details about OLRRP, including the entire network for Ontario and data documentations, can be found here.

Location: `/data/river_network/`

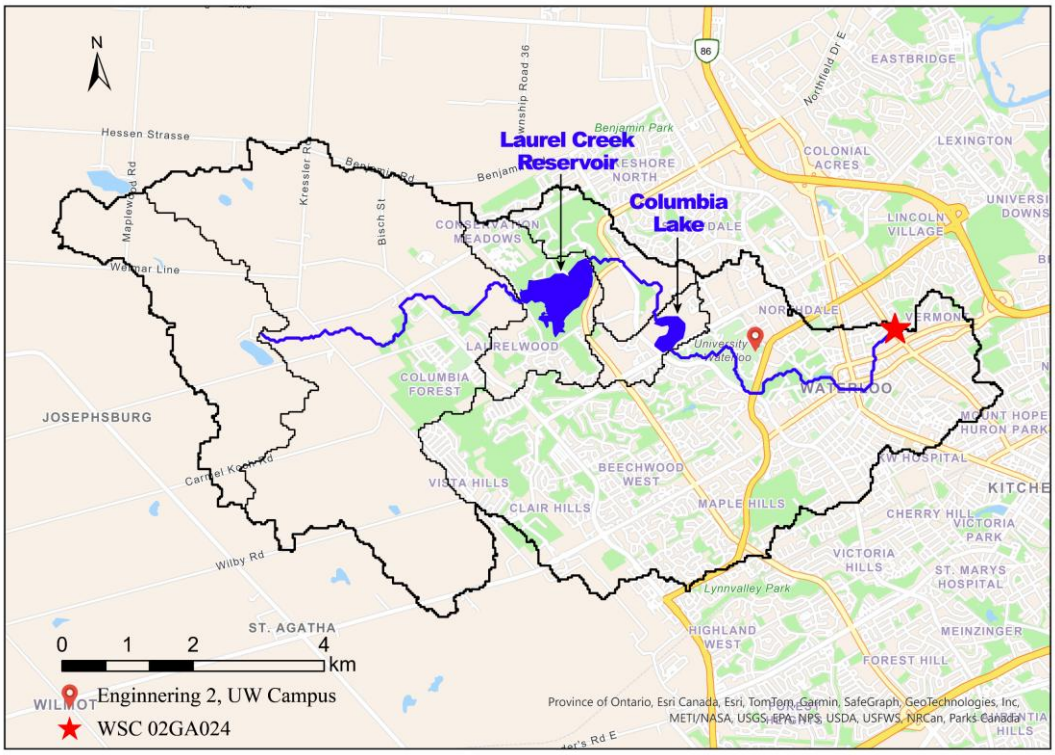


Figure 1. Laurel Creek catchment draining to WSC streamflow gauge 02GA024 (red star). The catchment boundary and subbasin discretization are shown, together with the river network and major waterbodies (Laurel Creek Reservoir and Columbia Lake). The location of the University of Waterloo (UW) campus (Engineering 2) is indicated by the pin. Basemap: Esri (World Navigation Map).

(2) Land cover (raster): Ontario Land Cover Version 1 (cropped)

Land cover is provided from the Ontario Land Cover Version 1 dataset (link here). The original dataset covers all of Ontario; for this lab, the raster has been **cropped to a rectangular region** that fully contains the study catchment. The raster resolution is **15 × 15 m**. Land cover class definitions are provided in the accompanying text file.

Location: `/data/land_cover/`

Metadata: `Baseline Class Descriptions - Ontario Land Cover Version 1.txt`

(3) Soil profile (vector): prepared dataset (use as-is)

The soil profile layer is provided as a ready-to-use vector dataset; no additional preprocessing is required in this walkthrough. A simplified soil scheme is used for the study area, containing two soil profile types: **DRAINED_AGRICULTURE_SOIL** and **DEFAULT_SOIL**.

Location: `/data/soil_profile/`

(4) Digital Elevation Model (raster): Ontario Provincial DEM (PDEM, cropped)

A DEM cropped from the Ontario PDEM (link here) is provided as a raster at **30 × 30 m** resolution. The DEM is used to compute mean elevation for HRUs and to derive slope and aspect for subsequent HRU attribute calculations.

Location: `/data/dem/`

(5) Two sets of Raven model files are provided for this lab.

- **Incomplete model file (to be completed)**

You will update this file by filling in the HRU information at the end of the lab.

Location: `/data/model_files/02GA024.rvh`

The channel_properties.rvp and Lakes.rvh files in this directory do **not** require any updates, use them as provided.

- **Benchmark (complete) model files (for verification only)**

Complete Raven model files are provided for (i) a **complex HRU configuration** that you will derive in this lab (v2-0-1a), and (ii) a **simple HRU configuration** (v2-0-1b; lake and non-lake HRUs only). Use these files **only** to verify your final outputs. Minor precision differences from geometric calculations are acceptable.

If you are to Section 4.4, you should use the `HRU_final` layer in `v2-0-1a` folder for Raven file preparation (Section 4.7), as this is a simplified HRU layer we provided for those skipped HRU simplification.

Location: `/data_benchmark/v2-0-1a/` and `/data_benchmark/v2-0-1b/`

2. Software and Tools

We use ArcGIS Pro for this walkthrough. ArcGIS Pro is only available on **Windows** (it does not run natively on macOS or Linux). The University provides a site license, so you may install and sign in ArcGIS Pro on your

own Windows machine or use the lab desktops. **ArcGIS Pro version 3.x** is suitable for this walkthrough.

We recommend **Visual Studio Code (VS Code)** for viewing end editing ASCII text files (link here). Notepad+ is compatible if you do not want to install VS code. **Microsoft Excel** can be used to view and edit tables. A PDF reader is also needed.

A pre-built ArcGIS toolbox, **HRU_simplify**, is provided with the lab materials. We will use it in this lab so that you do not need to build the workflow from scratch.

This walkthrough involves a range of ArcGIS Pro tools for data conversion, manipulation, and basic workflow automation. ArcGIS Pro tools are well documented, including definitions of inputs/outputs and parameter settings. When you are unsure how to run a tool or what an option means, consult the **official ArcGIS documentation** (link here). To keep the lab focused, we do not repeat full tool documentation here; instead, we emphasize the end-to-end workflow for HRU creation, HRU simplification, and attribute calculation.

Note: Software installation is not part of this walkthrough. Please ensure the required software is installed prior to the lab. A detailed guide for ArcGIS Pro installation is provided:

`/Docs/arcgis_download_instuctions.pdf`

3. Workflow Overview: HRU Discretization and Simplification

This walkthrough follows a two-stage workflow: (1) HRU discretization based on land cover, vegetation, soil, and lakes, and (2) HRU simplification to reduce small sliver polygons while preserving key hydrologic features.

3.1 HRU discretization (HRU_raw)

1) **Land cover reclassification**: The original land cover raster is reclassified using the provided mapping table (Location: `/Docs/landcover_reclassification_Feb2026.xlsx`) to produce a simplified set of land cover classes suitable for modeling.

2) **Raster-to-vector conversion and cleanup**: The reclassified land cover raster is converted to polygons and dissolved to remove internal boundaries and create clean, *discontiguous* land cover patches.

3) **Vegetation assignment**: A vegetation class is assigned to each land cover polygon using the same mapping table. A standalone vegetation layer is not created; instead, two vegetation-related fields are added to the land cover polygon attribute table and populated from the mapping.

4) **Union (landcover, vegetation & soil profile)**: Land cover/vegetation polygons are overlaid with the soil profile polygons using a **Union** operation to generate preliminary HRUs. Since we stored vegetation attributes in the land cover polygon, we only have two polygons in this operation. The resultant HRUs are coded as:

   `HRU_type = <land_ID><veg_ID><soil_ID>`

   where each ID is stored as a two-digit code (e.g., 00, 01, and 02). Example: land_ID = 02, veg_ID = 04, soil_ID = 01, then the HRU_type = 020401.

5) **Subbasin intersection**: The HRU layer is intersected with the subbasin boundaries, as such HRUs are explicitly defined **within each subbasin**.

6) **Lake override**: Lake polygons are used to define **lake HRUs**. Lake areas are then erased from the non-lake HRU layer to ensure *lakes override all other types*. Finally, lake HRUs and the erased HRUs are merged, and HRUs with the same type are dissolved within each subbasin to produce the initial HRU product (**HRU_raw**).

7) The **HRU_raw** attribute table should contain at least the eight fields listed below. All other fields created during intermediate processing steps (e.g., Union/Intersect outputs) or inherited from the subbasin layer are not needed for modeling and can be safely removed to keep the table clean.

```
land_ID (Long): Reclassified land cover class ID.
land_name (Text): Reclassified land cover class name.
veg_ID (Long): Vegetation class ID.
veg_name (Text): Vegetation class name.
soil_ID (Long): Soil profile class ID.
soil_name (Text): Soil profile class name.
HRU_type (Text): A six-character code formed by concatenating zero-padded two-digit IDs for land cover, vegetation, and soil:
HRU_type = <land_ID><veg_ID><soil_ID>
SubId (Text or Double): Unique subbasin ID inherited from the subbasin layer (after overlay operations).
```

Note: In addition to the eight modeling attributes listed above, ArcGIS will retain required system fields (e.g., ObjectID/FID and the Shape geometry field, and in many cases Shape_Area/Shape_Length). These system fields are managed by ArcGIS and should not be removed.

3.2 HRU simplification (HRU_final)

The HRU_raw layer often contains tiny polygons created by overlay operations. To reduce unnecessary complexity, we simplify HRUs using an **area threshold of 1.0 km^2**. Any *non-lake* HRU smaller than this threshold is classified as a *sliver*. Sliver HRUs are then merged into adjacent larger *non-lake* HRUs. A key rule in this simplification is that lake HRUs are never modified: no polygons are merged into lakes, and lake polygons are never merged into other HRU types.

1) **Separate lake and non-lake HRUs**: Lake HRUs are kept unchanged throughout simplification.

2) **Subbasin-level elimination of slivers**: Non-lake HRUs are processed **subbasin-by-subbasin** using the provided *HRU_simplify* toolbox. Any non-lake HRU smaller than **1.0 km^2** is merged into an adjacent larger HRU.

3) **Iterative cleanup**: The simplification is repeated if small slivers remain. In most cases, two rounds are sufficient to remove residual sliver HRUs.

4) **Merge**: The simplified non-lake HRUs are merged back with the unchanged lake HRUs to obtain the complete simplified HRU layer (**HRU_simp**). Assign each HRU a new field HRU_ID (Text), A unique HRU identifier created by concatenating the subbasin ID and HRU type:

```
HRU_ID = <SubId><HRU_type>
```

5) **Join subbasin attributes to HRUs.** Join the subbasin attribute table to the **HRU_simp** polygons using **SubId**. This ensures that each HRU carries the required subbasin-level information for downstream modeling and reporting. After the join, the attribute table should include all fields listed in Section 3.1, Step 7, plus the subbasin attributes from OLRRP. Finally, export the joined layer as a new feature class and name it **HRU_final** to permanently save the joined attributes.

3.3 HRU attribute calculation

For the **HRU_final** layer, we compute essential attributes commonly used in hydrologic modeling, including **HRU centroid coordinates (HRU_CenX/HRU_CenY)** and terrain statistics (**mean elevation, slope, and aspect**) derived from the DEM.

Note: DEM-based terrain statistics are optional in this walkthrough. As a simplified alternative, you may approximate these attributes using subbasin-level values, i.e., all HRUs within the same subbasin inherit the corresponding terrain attributes from the subbasin attribute table.

3.4 HRU file preparation for Raven modeling

We export the finalized HRU layer to shapefile and GeoJSON formats. We also use the HRU attributes (e.g., HRU_ID, SubId, area, land/soil/vegetation classes, and terrain attributes) to create or update a Raven RVH file, as such Raven can recognize HRUs and their properties during simulation.

4. Implementation Details (ArcGIS Pro)

This section summarizes key technical settings, field conventions, and recommended ArcGIS Pro tools used throughout the workflow presented in Section 3. These details help ensure results are reproducible and consistent across different computers.

4.1 Project setup and geoprocessing environments

If this is your first time creating an ArcGIS Pro project, save the **project file (.aprx)** in a location that is easy to find later. Note that the ArcGIS Pro project file and your datasets (shapefiles, rasters, feature classes, etc.) are stored *separately*: the .aprx file mainly preserves your project configuration such as maps, layer symbology, and layouts, and references the underlying data rather than embedding it. Recommended practice is to save the ArcGIS Pro project file (.aprx) in the same root directory as the `data/` folder, referred to as the `/working_folder/`.

All datasets in this lab (OLRRP routing network, land cover, soil profile, and DEM) have been prepared in the same **projected coordinate system**: **NAD 1983 Ontario MNR Lambert (EPSG:3161)** with linear units in **meters**. We are NOT using a **geographic coordinate system (GCS)** in this lab, since GCS based on degrees unit causes problems in calculating areas. Keep the ArcGIS Pro project and all geoprocessing outputs in EPSG:3161 to ensure consistent area/length calculations and to avoid misalignment during overlay operations.

In general, you should review the **Environment** settings before running any geoprocessing tool. Default options are often acceptable, but the following settings are recommended for this lab:

- **Output Coordinate System:** Use the default projected CRS: NAD 1983 Ontario MNR Lambert (EPSG:3161).
- **Processing Extent:** *Before* intersecting with the subbasin layer, you may use the default **rectangular extent** of the provided raster/soil datasets. *After* intersecting with the subbasin layer, set the extent to the **02GA024 catchment (subbasin/basin boundary)** to keep processing confined to the study area.
- **Snap Raster:** For any raster-based processing, set **Snap Raster** to the raster you want all outputs aligned to (typically the **land cover raster** or the **current input raster**).
- **Cell Size:** Match the **Snap Raster** (preferred) or the input raster cell size to ensure consistent resolution across outputs.

4.2 Land cover reclassification and vegetation fields

Inputs: The land cover raster and the mapping table:

`/Docs/landcover_reclassification_Feb2026.xlsx`

Use **Reclassify** tool from geoprocessing (directly search by name **from Analysis → Tools**). Define the land cover ID mapping as shown in the Excel table. For example, the original classes [21, 22, 23, 24, 25] are all mapped to FOREST with the new ID 3.

You can input the full mapping from the Excel table sheet, while only those existing land cover classes will be processed in ArcGIS. Some classes, e.g., urban, which do not exist in our region, will be safely skipped by ArcGIS.

Output: `landcover_reclass` raster

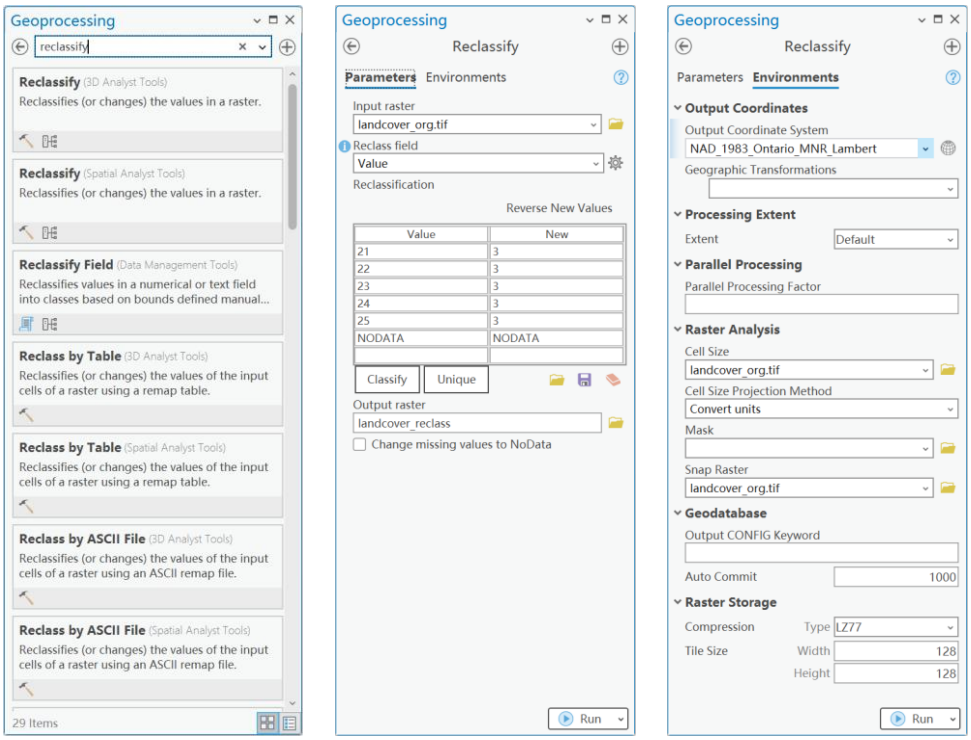


Figure 2. Example of searching for tools in geoprocessing and settings of Reclassify tool.

If anything went wrong after you click the **Run** button, check **Analysis → History** to review the error code.

After reclassification, use **Raster to Polygon** tool (Conversion Tools → From Raster → Raster to Polygon) to convert `landcover_reclass` to polygon features.

Input: `landcover_reclass` raster

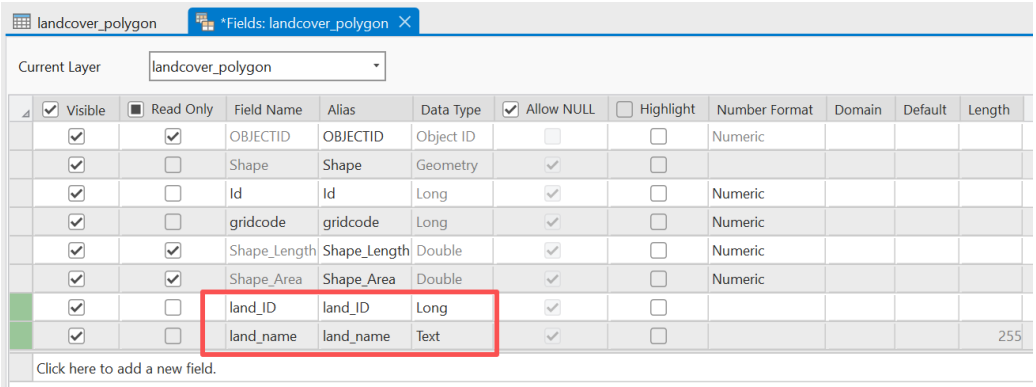
Output: `landcover_polygon` (to your geodatabase)

Simplify polygons should be **unchecked**. **Create multipart features** should be **checked**.

Right click the `landcover_polygon` layer from **Contents**, open its **Attribute Table**. Click Add to create new fields land_ID and land_name. See Figure 3.

After adding new fields, click Edit → Save to save changes.

Field naming note: Field names in ArcGIS have format-specific limits. File geodatabases support longer names (up to 64 characters), while **shapefiles** restrict field names to 10 characters and may truncate or rename fields during export. To avoid issues as we will use shapefile format to save HRUs in this lab, use clear names with letters/numbers/underscores (no spaces), avoid reserved/system names (e.g., Shape, ObjectID), and store coded identifiers such as HRU_type and HRU_ID as Text to preserve leading zeros. It is recommended to use the field names specified in this document to ensure consistency across the workflow.



Visible	Read Only	Field Name	Alias	Data Type	Allow NULL	Highlight	Number Format	Domain	Default	Length
✓	✓	OBJECTID	OBJECTID	Object ID			Numeric			
✓		Shape	Shape	Geometry	✓					
✓		Id	Id	Long	✓		Numeric			
✓		gridcode	gridcode	Long	✓		Numeric			
✓	✓	Shape_Length	Shape_Length	Double	✓		Numeric			
✓	✓	Shape_Area	Shape_Area	Double	✓		Numeric			
✓		land_ID	land_ID	Long	✓					
✓		land_name	land_name	Text	✓					255

Figure 3. Example of adding land_ID and land_name fields into the land cover attribute table.

Id and gridcode: They are created by ArcGIS. **Id** is an internal identifier for polygons, while **gridcode** stores the **input raster cell value** for each polygon. In this lab, Id can be safely deleted. And the gridcode values correspond to the reclassified land class IDs. To populate land_ID, right-click the land_ID field header and select **Calculate Field**. In the expression box, set: **land_ID = !gridcode!** (Figure 4). This can also be done **manually** by editing values directly in the attribute table. For example, we then can edit land_name row by row using land class names from the reclassification Excel table.

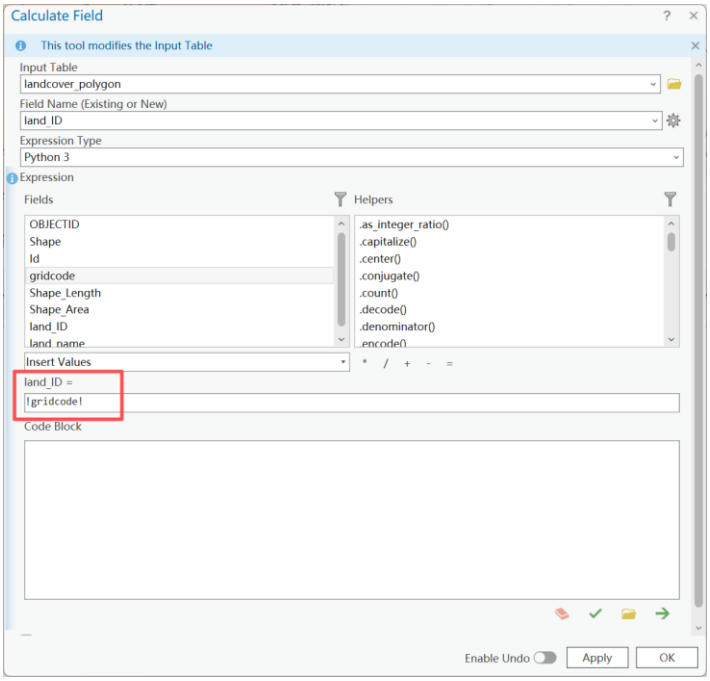


Figure 4. Example of using Calculate Field to assign values to a column based on other fields.

Similarly, add **veg_ID** and **veg_name** fields into the same attribute table to define vegetation attributes. Use the land cover-to-vegetation mapping provided in the reclassification Excel table to populate these fields. Remember to click **Edit → Save** to save changes.

4.3 HRU overlay operations

4.3.1 Union (land cover, vegetation, and soil profile)

Union (Analysis Tools → Overlay → Union) **is to** create preliminary HRU polygons by overlaying land/vegetation polygons with the soil profile layer.

Inputs: `landcover_polygon` (with land_ID, land_name, veg_ID, veg_name);

`soil_profile` (with soil_ID, soil_name)

Output: `HRU_union` (Each output polygon represents a unique spatial combination of land, veg and soil where boundaries overlap.)

After union operation, we **post-process `HRU_union`** and the resultant attribute table is shown in Figure 5:

- Delete any rows with missing land/vegetation/soil attributes, as these represent incomplete polygons at the input boundaries and are not useful for this workflow.
- **Water HRU soil override:** Identify HRUs with land_name = WATER. For these HRUs, set soil_name to **WATER** and update soil_ID to **3**. This assigns a dedicated WATER soil profile required by Raven. Using soil_ID = 3 avoids conflicts with the existing soil profile IDs in this lab.
- Remove unnecessary fields created by Union (e.g., FID_*, Id, and gridcode).
- Add and calculate HRU_type (Text).

Note: calculating HRU_type

Create a new Text field named HRU_type and use **Calculate Field** to generate the six-character code by concatenating **zero-padded two-digit** IDs. Right-click the HRU_type field header → **Calculate Field**, then use a Python expression such as:

```
HRU_type = f"{int(!land_ID!):02d}{int(!veg_ID!):02d}{int(!soil_ID!):02d}"
```

This will produce values like **030702** (land_ID=3, veg_ID=7, soil_ID=2).

HRU_union

Field: Add Calculate Selection: Select By Attributes Zoom To Switch Clear Delete Copy

	OBJECTID *	Shape *	land_ID	land_name	veg_ID	veg_name	soil_name	soil_ID	Shape_Leng	Shape_Length	Shape_Area	HRU_type
1	9	Polygon	1	WATER	1	NO_VEG	WATER	3	280819.947351	62556.41906	1587893.754388	010103
2	10	Polygon	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	405248.637011	14654949.228403	020201
3	11	Polygon	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	586698.524863	18049980.365171	030201
4	12	Polygon	5	URBAN	4	GRASSLAND	DEFAULT_SOIL	1	280819.947351	615630.324319	95705905.15267	050401
5	13	Polygon	6	AGRICULTURE	3	AGRICULTURE	DEFAULT_SOIL	1	280819.947351	909213.108014	65769637.917217	060301
6	14	Polygon	6	AGRICULTURE	3	AGRICULTURE	DRAINED_AGRICULTU...	2	241510.591482	241765.372779	17813396.247723	060302
7	15	Polygon	7	GRASSLAND	4	GRASSLAND	DEFAULT_SOIL	1	280819.947351	223934.86179	4692402.083125	070401

Click to add new row.

Figure 5. Example of `HRU_union` attribute table. WATER HRU is a special case (010103).

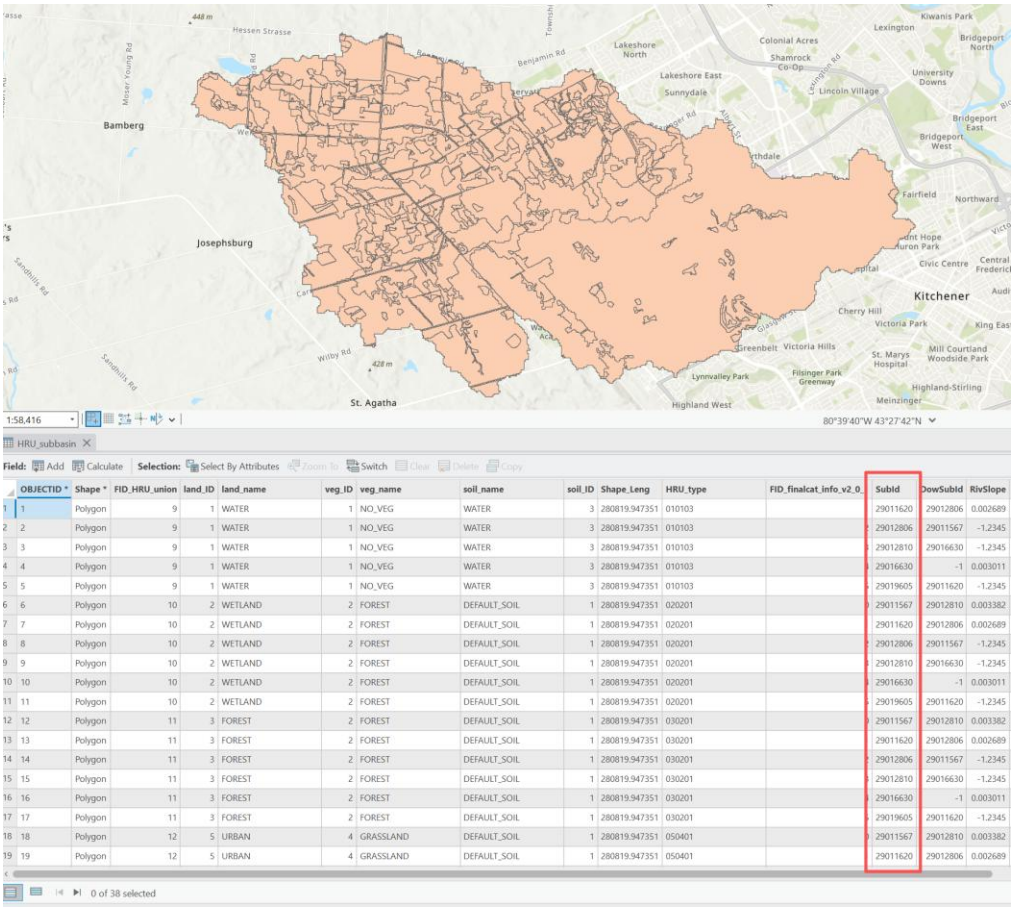
4.3.2 Intersect (`HRU_union` and subbasin polygons)

Intersect (Analysis Tools → Overlay → Intersect) is to ensure HRUs are defined within subbasins and carry the subbasin identifier. See Figure 6.

Inputs: `HRU_union` and subbasin polygon layer

`/data/river_network/finalcat_info_v2-0-1.shp`

Output: `HRU_subbasin` (Now every HRU polygon is associated with a specific SubId)



	OBJECTID *	Shape *	FID_HRU_union	land_ID	land_name	veg_ID	veg_name	soil_name	soil_ID	Shape_Leng	HRU_type	FID_finalcat_info_v2_0_	SubId	DowSubId	RivSlope
1	1	Polygon	9	1	WATER	1	NO_VEG	WATER	3	280819.947351	010103		29011620	29012806	0.002689
2	2	Polygon	9	1	WATER	1	NO_VEG	WATER	3	280819.947351	010103		29012806	29011567	-1.2345
3	3	Polygon	9	1	WATER	1	NO_VEG	WATER	3	280819.947351	010103		29012810	29016630	-1.2345
4	4	Polygon	9	1	WATER	1	NO_VEG	WATER	3	280819.947351	010103		29016630	-1	0.003011
5	5	Polygon	9	1	WATER	1	NO_VEG	WATER	3	280819.947351	010103		29019605	29011620	-1.2345
6	6	Polygon	10	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	020201		29011567	29012810	0.003382
7	7	Polygon	10	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	020201		29011620	29012806	0.002689
8	8	Polygon	10	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	020201		29012806	29011567	-1.2345
9	9	Polygon	10	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	020201		29012810	29016630	-1.2345
10	10	Polygon	10	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	020201		29016630	-1	0.003011
11	11	Polygon	10	2	WETLAND	2	FOREST	DEFAULT_SOIL	1	280819.947351	020201		29019605	29011620	-1.2345
12	12	Polygon	11	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	030201		29011567	29012810	0.003382
13	13	Polygon	11	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	030201		29011620	29012806	0.002689
14	14	Polygon	11	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	030201		29012806	29011567	-1.2345
15	15	Polygon	11	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	030201		29012810	29016630	-1.2345
16	16	Polygon	11	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	030201		29016630	-1	0.003011
17	17	Polygon	11	3	FOREST	2	FOREST	DEFAULT_SOIL	1	280819.947351	030201		29019605	29011620	-1.2345
18	18	Polygon	12	5	URBAN	4	GRASSLAND	DEFAULT_SOIL	1	280819.947351	050401		29011567	29012810	0.003382
19	19	Polygon	12	5	URBAN	4	GRASSLAND	DEFAULT_SOIL	1	280819.947351	050401		29011620	29012806	0.002689

Figure 6. Example of the `HRU_subbasin` layer and its attribute table after the **Intersect** operation with the 02GA024 subbasin layer. Each HRU polygon is assigned a subbasin identifier via the **SubId** field (highlighted), and the corresponding subbasin attributes are propagated to the HRU features.

4.3.3 Erase (Lake override)

In the land cover and soil profile datasets, lakes are not represented explicitly. Therefore, we assign a dedicated **LAKE** HRU code by setting the land, vegetation, and soil IDs to 0 (i.e., **land_ID = 0, veg_ID = 0, soil_ID = 0**) for all lake HRUs.

Lake polygons are provided in the lake layer (`data/river_network/sl_connected_lake_v2-0-1.shp`). In this catchment, there are only two lake polygons. We use the **Erase** tool (Analysis → Overlay → Erase) to remove lake areas from `HRU_subbasin`, which creates two holes in the non-lake HRU layer. These holes are then filled by appending (merging) the lake HRU polygons. All these processes are to ensure that lakes override all other HRU types.

Input: `HRU_subbasin`

Erase feature: lakes from `sl_connected_lake_v2-0-1`

Output: `HRU_nolake`

Note #1: creating `HRU_lake`

The original lake layer `sl_connected_lake_v2-0-1` is used only as a lake footprint. Export this layer to a new feature class named `HRU_lake`, and then populate the lake HRU attributes. For lake HRUs, keep only the required fields: **land_ID, land_name, veg_ID, veg_name, soil_ID, soil_name, and HRU_type**. Assign a dedicated lake code by setting land_ID = 0, veg_ID = 0, and soil_ID = 0 (with names set to **LAKE**), and calculate HRU_type accordingly (i.e., **000000** for a zero-padded two-digit scheme).

Note #2: adding SubId to `HRU_lake`

SubId (subbasin ID) is a required attribute for the HRU layer and already exists in `HRU_nolake`. Before merging lakes back in (Step 4.3.4), you must also populate SubId for `HRU_lake`. *Any* of the following approaches is acceptable:

(i) Manual edit:

Add a SubId field to `HRU_lake` (use the same field type as in `HRU_nolake`). Then, using the subbasin layer `finalcat_info_v2-0-1`, identify which subbasin each lake belongs to and manually enter the SubId values. This is feasible here since there are only two lakes.

(ii) Spatial overlay (Intersect):

Use the **Intersect** tool to intersect `HRU_lake` with `finalcat_info_v2-0-1`. The output will inherit SubId from the subbasin layer based on spatial overlap.

(iii) Attribute join (most efficient):

Join SubId (and other subbasin attributes) from the subbasin layer to `HRU_lake` using **lake IDs**:

- Right-click `HRU_lake` → Joins and Relates → Add Join
- Input Table: `HRU_lake`
- Input Join Field: Hylak_id
- Join Table: `finalcat_info_v2-0-1`
- Join Table Field: HyLakeId
- Check Keep All Target Features
- Leave Index Joined Fields unchecked
- **Run the join and export features (e.g., `HRU_lake_join`) to permanently save the joined SubId**

Note: **Hylak_id** and **HyLakeId** both represent unique lake identifiers (with different field names), so they can be safely used as join keys.

4.3.4 Merge and Dissolve (HRU_raw)

After the previous step, you should have `HRU_lake` (or `HRU_lake_join` if you used option (iii) to populate SubId in Step 4.3.3) and `HRU_nolake`. You can now combine these two layers to form a complete HRU layer using the **Merge** tool (Data Management Tools → General → Merge). See Figure 7 for the key settings.

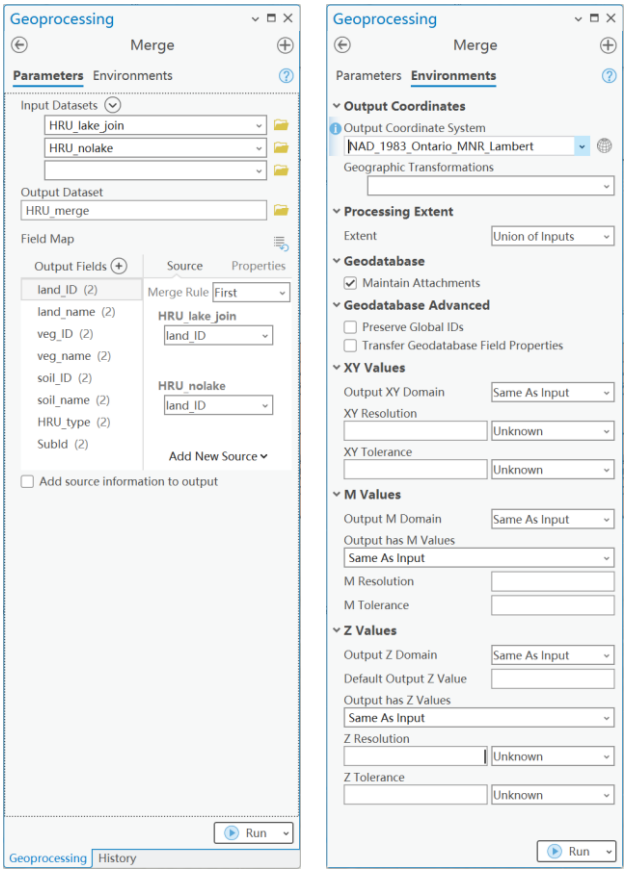


Figure 7. Example of **Merge** settings for combing `HRU_lake_join` and `HRU_nolake`. For a clean attribute table in the `HRU_merge`, we should delete all useless fields in the Output Fields and only keep the eight necessary fields specified in Section 3.1.

In `HRU_merge`, HRUs of the same type within a subbasin may still appear as multiple separate polygons. To merge these into *discontiguous* multi-part patches, run the Dissolve tool (Data Management Tools → Generalization → Dissolve) using SubId and HRU_type as the dissolve fields. Output is `HRU_raw`.

If you would like to retain descriptive attributes in the output table, you may also include land_ID, land_name, veg_ID, veg_name, soil_ID, and soil_name as additional dissolve fields. These fields will not change the dissolve grouping since they are already uniquely implied by HRU_type.

Now we add HRU_area field to the `HRU_raw`.

- Add a new field HRU_area (Double) to `HRU_raw`.
- Right-click HRU_area → **Calculate Geometry** and compute area in km^2 or m^2.

After the dissolve operation, subbasin attributes are gone from `HRU_raw`. If you will proceed with Section 4.4 (HRU simplification), subbasin attributes will be re-joined to HRU table in later processing. However, **if you are to skip Section 4.4 and directly go into Section 4.5, you must add subbasin attributes back to the `HRU_raw` now,** since they are required in Section 4.5. Follow the approach below to join attributes.

Note: adding subbasin attributes to `HRU_raw` (for those skip Section 4.4)

Join subbasin-level information (e.g., drainage area, river length, slope, routing parameters, or any precomputed basin attributes) to each HRU polygon so every HRU inherits the attributes of its parent subbasin.

Ensure both layers contain a matching key field:

- HRU layer key: SubId
- Subbasin layer/table key: SubId

In the **Contents** pane, right-click `HRU_raw` → **Joins and Relates** → **Add Join**.

- Input Table: `HRU_raw`
- Input Join Field: SubId
- Join Table: subbasin layer (`finalcat_info_v2-0-1`)
- Join Table Field: SubId

Click **Run** and verify in the attribute table that subbasin fields are appended to `HRU_raw`. Confirm the join match rate is 100% (no NULLs in key joined fields). Spot-check a few HRUs to ensure they inherited the correct subbasin attributes.

Save the result (important)! Joins are temporary. To permanently store the joined attributes: Use **Data** → **Export Features** to create a new feature class, e.g., `HRU_raw_join`.

If you will skip Section 4.4 proceed with `HRU_raw_join` layer and use it in Section 4.5.

If you proceed with Section 4.4, above join is not required here, so you still use `HRU_raw` in section 4.4.

The **`HRU_raw`** (**or `HRU_raw_join`**) layer represents the **finest** spatial resolution supported by the available datasets. At this stage, you may explore the layer to better understand the HRU patterns. For example, you can (i) adjust the color scheme to visualize different HRU types (Layer → right-click → Symbology), (ii) inspect field statistics to assess spatial heterogeneity (right-click a field header → Statistics), and (iii) add an HRU_area field and compute polygon areas (right-click a field header → Calculate Geometry) to identify small polygons. These steps are *optional*, but you are encouraged to try them if time allows.

4.4 HRU simplification (Sliver elimination)

* This section is OPTIONAL for course ENVE 383.

After completing the overlay operations, the **HRU_raw** layer often contains narrow "sliver" polygons created by raster-to-polygon conversion and overlay boundaries. These slivers increase the number of HRUs and can significantly increase model complexity without adding meaningful hydrologic information. In this lab, we simplify HRUs using an **area threshold of 1.0 km^2**.

Sliver definition: Any **non-lake HRU** with area < 1.0 km^2 is considered a sliver HRU.

Key rule: Lake HRUs are never modified, i.e., no polygons are merged into lakes, and lake polygons are never merged into other HRU types even though they are smaller than the sliver area threshold.

4.4.1 Identify slivers based on HRU_area

- Identify non-lake sliver HRUs with HRU_area < 1.0 (if area is in km^2).
- If you did not see any slivers, then you are good to go into Section 4.5. If you identified slivers, proceed with Section 4.4.2.

Tip: You can use the attribute table **Statistics** view to quickly check the distribution of HRU sizes.

4.4.2 Simplify HRU subbasin by subbasin

Why simplify per subbasin?

HRU simplification uses neighbor-based merging (**Eliminate**). If we simplify the entire basin at once, polygons near subbasin boundaries may merge across subbasins, which is not allowed. Processing one subbasin at a time ensures:

- All merges stay inside the same subbasin;
- Results are easier to debug and validate;
- The workflow can be **repeated** for all subbasins in a consistent way.

The HRU simplification workflow relies primarily on two geoprocessing operations: **Select By Attributes** (to identify sliver polygons) and **Eliminate** (to merge slivers into adjacent larger HRUs). Per-subbasin HRU simplification is inherently *iterative* in two ways: (1) the same procedure must be repeated for each subbasin, and (2) for a given subbasin, the procedure may need to be run multiple times (e.g., two rounds) until all candidate sliver polygons are removed. For this reason, we build a reusable workflow that *chains* the required tools together and then repeatedly run it to produce the final HRU layer.

In the `HRU_raw` attribute table, use **Select By Attributes** to select all lake HRUs where **HRU_type = "000000"**, then export the selected features as `HRU_raw_lake` (we have 2 lake HRUs). Similarly, select all non-lake HRUs (i.e., **HRU_type <> "000000"**) and export them as `HRU_raw_nolake` (we have ~38 non-lake HRUs). We simplify only the **non-lake** HRUs, and keep lake HRU feature untouched.

We split subbasins first. Open **Split By Attributes** (Analysis → Tools → Split By Attributes).

Input Features: `HRU_raw_nolake`

Split Field: SubId

Output Workspace: select current geodatabase, such as

`/working_folder/MyProject8/MyProject8.gdb`

Expected output: a set of feature classes, one per subbasin (e.g., T<SubId>_0), each containing only the non-lake HRUs for that subbasin. Output features are viewable through the Catalog pane, and they will not be loaded to the map by default. See Figure 8.

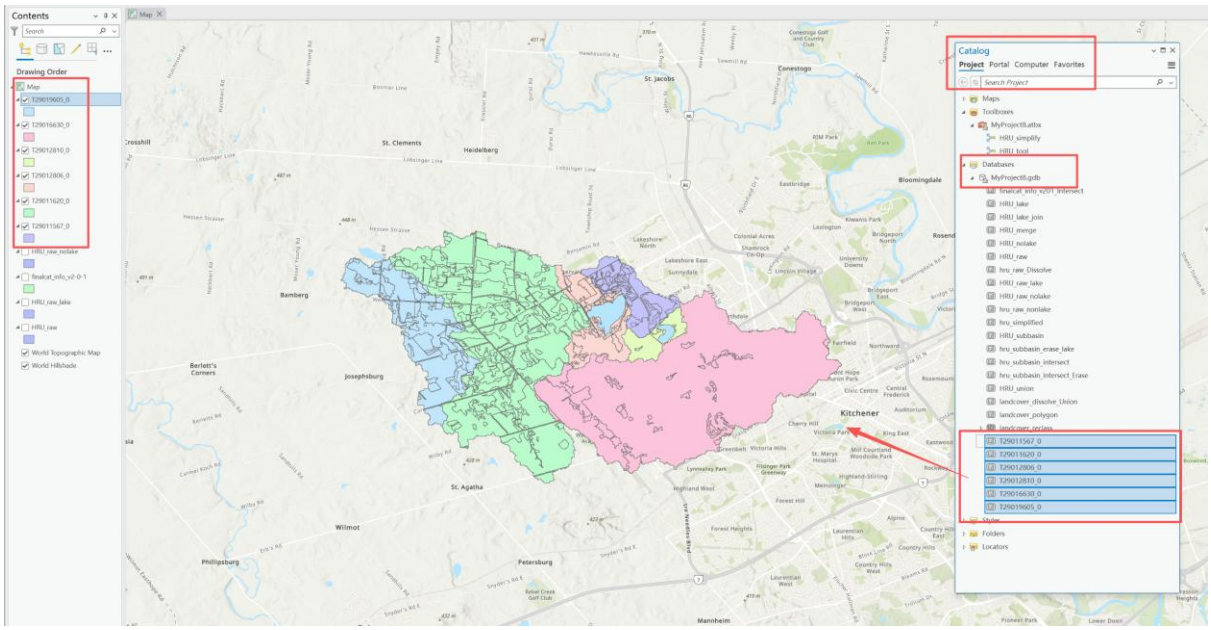


Figure 8. Example of viewing the per-subbasin HRU feature classes created by **Split By Attributes** in the **Catalog** pane. These split outputs follow the naming pattern **T*_0** and can be dragged directly from Catalog onto the map to visualize each subbasin subset.

In this walkthrough, we provide a *pre-built* **ModelBuilder** tool, **HRU_simplify**, located at

`/working_folder/HRU_simplify_tool.atbx`

You can add this toolbox to ArcGIS Pro and run the tool directly, without building the model from scratch. Before running, please **update the input parameters** (and, if needed, the output workspace) so they point to **your own geodatabase**.

Note: You are encouraged to try the full procedure to build HRU_simplify (estimate within 1 hour). A step-by-step guide is provided in **Appendix A**.

Follow steps below to use the pre-built HRU Simplification Tool (HRU_simplify):

Step 1. Load the toolbox into ArcGIS Pro

Open your ArcGIS Pro project. In the **Catalog** pane, right-click **Toolboxes → Add Toolbox**.

Browse to `/working_folder/HRU_simplify_tool.atbx` and add it.

Right-click it and select **Edit** HRU_simplify. See Figure 9. The model consists of several key tools:

- **Iterate Feature Classes:** Loop the split subbasins in the input geodatabase (see Figure 8).
- **Select by Attribute:** Identifies slivers using the user-defined threshold (**Sliver_threshold_km2**).
- **Eliminate:** Merges the selected slivers to produce a simplified HRU layer.
- **Output:** Saves the result with a standardized name (e.g., **%BaseName%_run*N***) for easy comparison across runs.

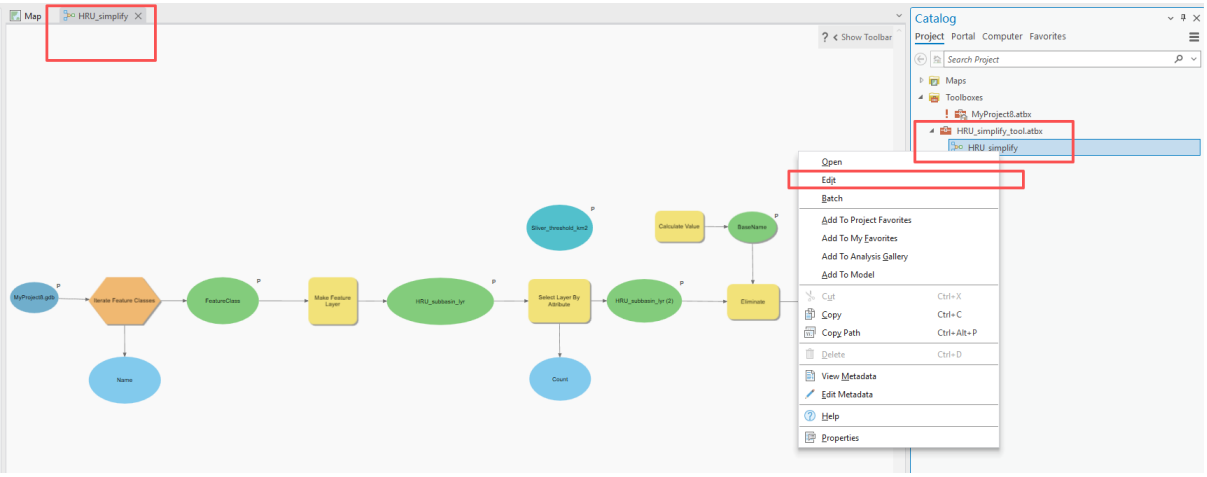


Figure 9. Opening the pre-built HRU_simplify ModelBuilder tool. In the Catalog pane, locate HRU_simplify_tool.atbx, expand it, and right-click HRU_simplify to Open (or Edit) the model. The model will open in a new HRU_simplify tab, where you can review the workflow and update input paths before running.

Step 2: Before you run the tool

Review each input parameter and ensure it points to **your own geodatabase** (paths may differ from the example project).

- Double-click **Iterate Feature Classes**.

  Workspace: Set this to your geodatabase. (By default, it is MyProject8.gdb on Hongren's machine. You must change it to yours.)

  Wildcard: Set this based on your split subbasin naming pattern. For example, in Figure 8 our subbasins are named **T<SubId>_0**, so we use **T*_0** as the wildcard. The asterisk (*) matches the variable part of the name during looping.
- Double-click **Select Layer By Attribute**.

  Review the expression: `HRU_area < %Sliver_threshold_km2%`
- Double-click **Sliver_threshold_km2**. Default value is 1.
- Double-click **Calculate Value**.

  Expression: `clean_base(r"%Name%")`

  Code Block:

```
import re

def clean_base(s):

    s = "" if s is None else str(s)

    return re.sub(r'_run\d+$', '', s)
```

- Double-click **Eliminate**.

  Name of Output Feature Class: (…) /**`MyProject8.gdb/%BaseName%_run1`**

  Make sure `%BaseName%_run1` is in the same geodatabase with inputs to keep results organized.

Step 3: Run (First round)

Run HRU_simplify and check the per-subbasin output in the Catalog pane (Figure 10).

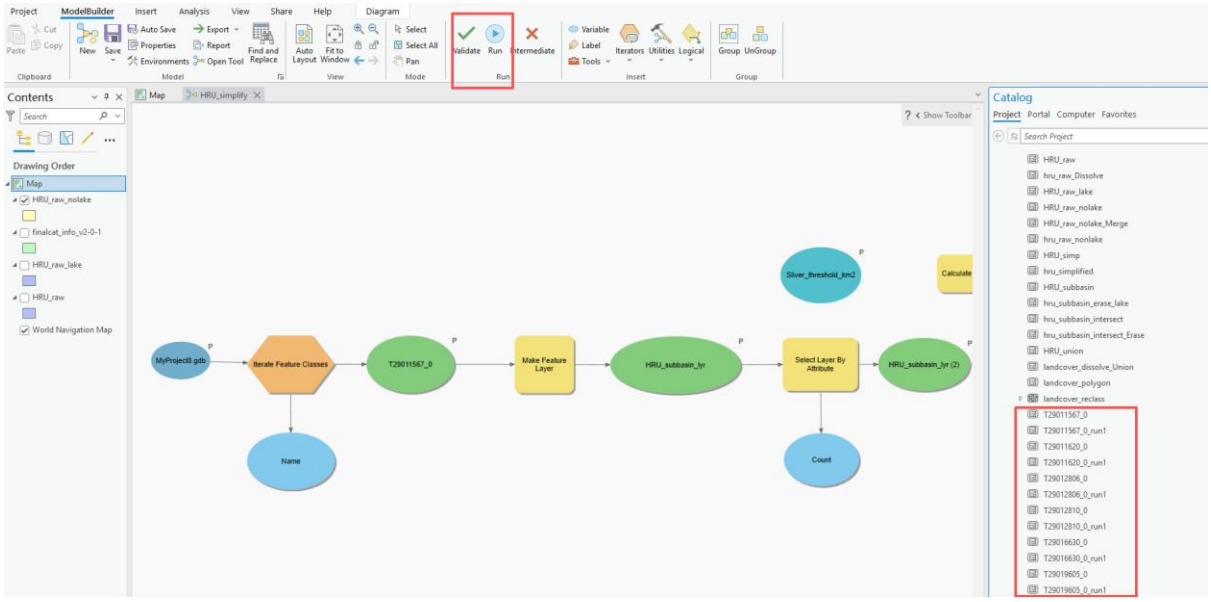


Figure 10. Running the HRU_simplify model and verifying outputs. Click Run in ModelBuilder to execute HRU_simplify (top). After completion, confirm that simplified per-subbasin feature classes (e.g., T<SubId>_0_run1) are created in the project geodatabase and listed in the Catalog pane (right).

Termination rule

Visually inspect the result to confirm that small sliver HRUs are reduced as expected. Load the T<SubId>_0_run1 layers to **Map** and check their areas.

Note that HRU_area field may be outdated if an HRU was changed in run1. **You should either recalculate this field or use Shape_Area (m^2) for sliver checking.** Shape_Area is automatically maintained by ArcGIS.

In this example, we need a second-round elimination.

Step 4: Run (Second round)

For the second-round run, use the first-round outputs as the new inputs, and save results with a different output

name (e.g., a new run suffix). Note that you can still include subbasins that no longer contain slivers, as those feature classes will remain unchanged, and the model will simply continue the loop.

- Double-click **Iterate Feature Classes**.
  Workspace: Set this to your geodatabase. Keep it as-is given we have set it correctly in Step 3.
  Wildcard: **T*_0_run1** is an acceptable form.
- Double-click **Eliminate**.
  Name of Output Feature Class: `(…)/MyProject8.gdb/%BaseName%_run2`

Run HRU_simplify and check the per-subbasin output in the Catalog pane. Now we should have new per-subbasin outputs with names T<SubId>_0_run2.

Step 5. Merge results

After *N* rounds of iteration, we get rid of all sliver HRUs. The final outputs T*_0_run*N* from Run *N* can then be merged to form a basin-wide simplified HRU layer. Use the **Merge** tool (Data Management → General → Merge) to combine all T*_0_run*N* feature classes, and include `HRU_raw_lake` in the merge to reintroduce lake HRUs (Figure 11). **Update HRU_area field after merging.**

The resulting basin-wide layer (`HRU_simp`) contains 14 HRUs. The two lake HRUs were not modified during simplification and are carried through unchanged.

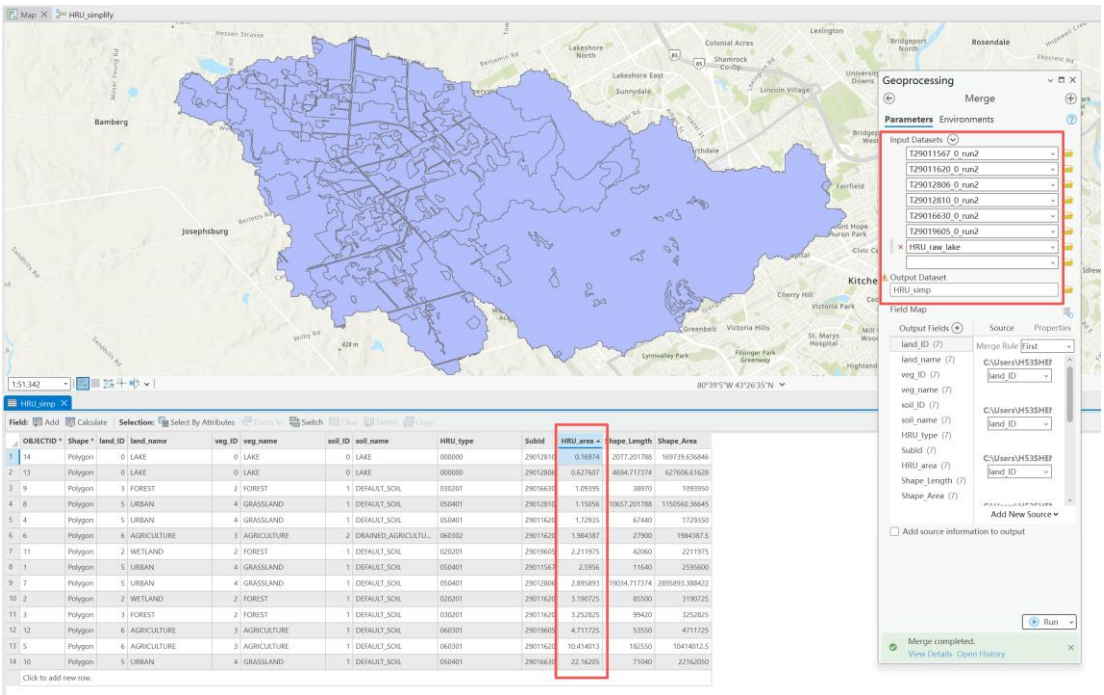


Figure 11. Merging simplified subbasin HRU outputs to create a basin-wide HRU layer `HRU_simp`. All T*_0_run2 feature classes from Round 2 are merged, and `HRU_raw_lake` is included to reintroduce lake HRUs. Ensure the **Merge** Field Map includes matching fields from all inputs so HRU attributes (e.g., land/veg/soil IDs, HRU_type, SubId, HRU_area) are preserved.

4.4.3 Add HRU_ID (unique HRU identifier)

After generating the basin-wide HRU polygon layer (`HRU_simp`), create a new field called **HRU_ID** to assign a **unique Text ID** to every HRU polygon. This ID is used as the primary key for modeling. Build HRU_ID from the subbasin ID plus HRU_type:

```
HRU_ID = <SubId><HRU_type>
```

For example, if SubId = 29012810 and HRU_type = 030201, then HRU_ID = 29012810030201

Use the following **Calculate Field (Python) expression** to populate HRU_ID as a text identifier by concatenating the subbasin ID (SubId) and the HRU class code (HRU_type):

```
str(int(!SubId!)) + str(!HRU_type!)
```

4.4.4 Join subbasin attributes to HRU layer

Join subbasin-level information (e.g., drainage area, river length, slope, routing parameters, or any precomputed basin attributes) to each HRU polygon so every HRU inherits the attributes of its parent subbasin.

Ensure both layers contain a matching key field:

- HRU layer key: SubId
- Subbasin layer/table key: SubId

In the **Contents** pane, right-click `HRU_simp` → **Joins and Relates** → **Add Join**.

- Input Table: `HRU_simp`
- Input Join Field: SubId
- Join Table: subbasin layer (`finalcat_info_v2-0-1`)
- Join Table Field: SubId

Click **Run** and verify in the attribute table that subbasin fields are appended to `HRU_simp`. Confirm the join match rate is 100% (no NULLs in key joined fields). Spot-check a few HRUs to ensure they inherited the correct subbasin attributes.

Save the result (important)! Joins are temporary. To permanently store the joined attributes: Use **Data** → **Export Features** to create a new feature class, e.g., `HRU_final`.

4.5 HRU attributes calculation

In this step, we populate the HRU attributes required for modeling in the basin-wide HRU polygon layer (**`HRU_final` or `HRU_raw_join` if you skipped Section 4.4**). HRU area was already calculated in Section 4.4, so here we focus on (1) HRU centroid (or central point) coordinates and (2) terrain attributes (mean elevation, slope, and aspect). We will use the following field names **as-is**:

- HRU_CenX (for Lon), HRU_CenY (for Lat) (centroid X/Y or central point X/Y)
- HRU_E_mean (mean elevation)
- HRU_S_mean (mean slope)
- HRU_A_mean (mean aspect)

Open the attribute table of `HRU_final` (or `HRU_raw_join` if you skipped Section 4.4) and add the following fields (type **Double**):

Use **Calculate Geometry Attributes** to write centroid (or central point) coordinates directly into the HRU polygon layer. Note that since we want coordinates to be based on degree unit, use **NAD 1983 or WGS 1984** as the coordinate system in this calculation.

Mean elevation, slope, and aspect are commonly derived from a DEM. We provide a DEM for this basin, but DEM-based processing is *optional* in this lab. Instead, we use **subbasin-based terrain attributes as a substitute**, which are already available as subbasin attributes were joined to `HRU_final` in Section 4.4.4 or `HRU_raw_join` layer in Section 4.3.4 if you skipped Section 4.4.

Use **Calculate Field** to populate the HRU terrain fields from the joined subbasin fields:

- HRU_E_mean ← MeanElev
- HRU_S_mean ← BasSlope
- HRU_A_mean ← BasAspect

Note: With this approach, all HRUs within the same subbasin share the same elevation/slope/aspect values. This is sufficient for the lab exercise. For real modeling applications, DEM-based HRU statistics are recommended.

4.6 HRU QAQC

After calculating HRU attributes (Section 4.5), perform a quick QA/QC check to ensure the final HRU layer (**`HRU_final` or `HRU_raw_join` if you skipped Section 4.4**) is complete and ready for export and modeling.

Geometry checks

Visually inspect `HRU_final` or `HRU_raw_join` to confirm HRU polygons cover the basin as expected (no obvious gaps or overlaps). Confirm **lake HRUs are present and unchanged** (two lake HRUs should still exist).

Reality check against an Esri basemap

To ensure HRUs and their attributes look realistic, compare them with an Esri basemap in ArcGIS:

In the **Map** tab, open **Basemap** and add **Imagery Hybrid Map**. Turn on HRU_final and set the layer to **hollow fill** (outline only) so the imagery is visible. Spot-check several locations:

- HRUs labeled as "forest/agriculture/urban" (based on the land cover class) should roughly align with the imagery patterns.
- Lake HRU boundaries should match visible water bodies.
- If large mismatches are found, revisit earlier steps (e.g., land cover reclassification, lake handling, or elimination rules) before exporting.

Attribute completeness checks

Open the attribute table and verify key fields are populated (no unexpected NULLs):

- IDs and classes: HRU_ID, SubId, HRU_type
- Centroids: HRU_CenX, HRU_CenY
- Terrain: HRU_E_mean, HRU_S_mean, HRU_A_mean
- Area field from Section 4.4 (HRU_area): must be recalculated when simplification is done.

Uniqueness checks

Verify HRU_ID is unique (no duplicates).

Range/sanity checks

- HRU_CenX and HRU_CenY should be reasonable (degrees if you used WGS84/NAD83).
- HRU_E_mean should be within realistic elevation ranges for the basin.
- HRU_S_mean should be non-negative and within a realistic slope range.
- HRU_A_mean should be between 0 to 360 (aspect is in degrees).

These QA/QC steps help catch common issues such as incorrect joins, missing lake protection, or misclassified land cover before the final export.

4.7 HRU export (Shapefile and GeoJSON)

After Section 4.6, export the final HRU layer (**`HRU_final` or `HRU_raw_join` if you skipped Section 4.4**) to both shapefile and GeoJSON formats.

In the **Contents** pane, right-click `HRU_final` **(**or `HRU_raw_join` if you skipped Section 4.4**)** → Data → Export Features. Set the output location to a folder and name the output as `HRU_final.shp`.

Do not export to a geodatabase. Choose a regular folder as the output location, e.g., `/working_folder/HRU_final.shp`

To export GeoJSON, run **Features To JSON** (Conversion Tools → JSON → Features To JSON). Set:

Input Features: `HRU_final` **(**or `HRU_raw_join` if you skipped Section 4.4**)**

Output JSON File: `HRU_final.geojson` (e.g., `/working_folder/HRU_final.geojson`)

Format JSON: **checked** (optional, for readability)

Output to GeoJSON: **checked**

(**MUST-DO**) Set Output Coordinate System to WGS 1984 (EPSG:4326). **Project to WGS_1984: checked**.

Notes for shapefiles

- A shapefile is stored as multiple files (at least **.shp, .shx, .dbf, .prj**; sometimes .cpg). Keep them together in the same folder.
- Field names are limited to **10 characters**. Longer field names may be truncated automatically.
- Attribute storage is limited (DBF table): long text, very large numbers, and high-precision decimals may be rounded or truncated.
- Shapefiles can only store one geometry type per file (here: polygons).
- Re-add `HRU_final.shp` into ArcGIS Pro and quickly check the attribute table (e.g., HRU_ID, SubId, HRU_type, HRU_CenX, HRU_CenY, HRU_E_mean, HRU_S_mean, HRU_A_mean) to confirm the export is correct.
- GIS software and many programming languages can read it easily, e.g., ArcGIS, QGIS, Python with `GeoPandas`, and R with `sf`.

Notes for GeoJSON

- GeoJSON is stored as a single file (**.geojson**), which is convenient for sharing and for web-based visualization. You can open it in VS Code.
- GeoJSON is typically expected in WGS 1984 (EPSG:4326) (longitude/latitude in degrees). When exporting, set the output coordinate system to WGS 1984.
- GeoJSON does **not** have the shapefile 10-character field name limit, so attribute field names are preserved.
- For large HRU datasets, GeoJSON may be slower to load than a geodatabase; use it mainly for sharing/visualization.
- RavenView supports GeoJSON. RavenView can load GeoJSON files for visualization. It requires GeoJSON in WGS84 for displaying the map correctly.

5. Raven HRU File Preparation (RVH)

If skipped Section 4.4, you should use the provided simplified HRU shapefile in this section.

Location: `/data_benchmark/v2-0-1a/HRU_final.shp`

If you have done simplification in Section 4.4, use your own `HRU_final` layer in this section.

Raven does not read HRU shapefiles directly. Instead, we must convert the `HRU_final` attribute table into the ASCII format required by Raven (an RVH file). In this section, we first prepare the HRU attribute table by harmonizing column names in Excel, and then insert the HRU records into a Raven RVH file.

To Export `HRU_final` attribute table to CSV, open the attribute table, then click the table options menu (top-right icon; Figure 12) → Export. In the export dialog, choose a regular folder as the output location, e.g.,

`/working folder/HRU_final.csv`

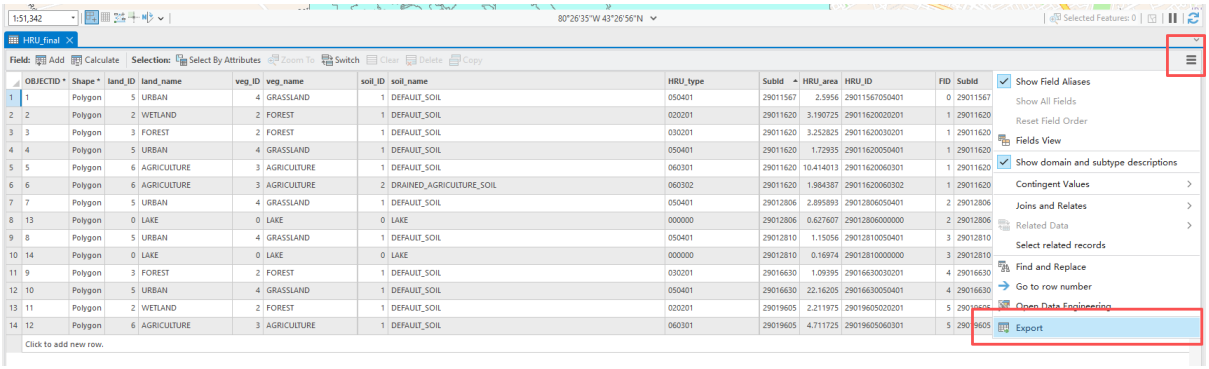


Figure 12. Exporting the `HRU_final` attribute table to a CSV file using **Export** (highlighted).

Note: If you have an active selection, the attribute table will indicate that records are selected, and only those selected HRUs may be exported. To avoid this, make sure nothing is selected before exporting by clicking the blank square above row #1 (**top-left corner of the table**) to clear any selection.

Then, edit the CSV table in Excel and reorganize fields for Raven RVH. Open the exported `HRU_final.csv` in **Microsoft Excel**. **Save As** an Excel workbook (recommended), e.g., `HRU_final.xlsx`, so you can safely rename/reorder columns.

Create a **Raven-ready subset table** by keeping only the fields required by Raven for the **:HRUs** block in RVH file. Here, we attached Raven manual (v4.1) in our material:

`/Docs/RavenUsersManual_v4.1.pdf`

Go to **page 206-207** in the manual for details of HRU requirements.

Raven's expected fields include:

`ID, AREA, ELEVATION, LATITUDE, LONGITUDE, BASIN_ID, LAND_USE_CLASS, VEG_CLASS, SOIL_PROFILE, AQUIFER_PROFILE, TERRAIN_CLASS, SLOPE, and ASPECT`

In the `HRU_final.xlsx` sheet, all required fields are already included, although some may use different field names. In this step, you do not need to rename fields. You only select the required columns and reorder them to match the Raven RVH field order listed above. Note that some columns may be stored as numeric values and displayed with decimal places (e.g., HRU_ID and SubId). Make sure these ID fields are formatted as whole numbers (no decimals). You may also adjust the number of decimal places for other numeric fields as needed.

After editing, copy the table (**without headers**) and paste it into RVH file (Figure 13). You may add extra spaces between columns to improve **alignment and readability**. Raven treats any whitespace as a delimiter, so aligning columns with spaces will not affect model parsing.

```
:SubBasinProperties
  :Parameters      Q_REFERENCE     TIME_CONC     TIME_TO_PEAK
  :Units           none            none          none
  29011567         5.325830        0.045500      0.013650
  29011620         4.356320        0.099918      0.029975
  29012806         4.914580        0.051104      0.015331
  29012810         5.535020        0.035193      0.010558
  29016630         9.219710        0.104685      0.031406
  29019605         1.096990        0.066059      0.019818
:EndSubBasinProperties

:HRUs
  :Attributes      AREA            ELEVATION     LATITUDE      LONGITUDE     BASIN_ID      LAND_USE_CLASS  VEG_CLASS     SOIL_PROFILE              AQUIFER_PROFILE  TERRAIN_CLASS  SLOPE         ASPECT
  :Units           km2             m             deg           deg           none          none            none          none                      none             none           deg           deg
  2901156705040 1  2.5956000000    345.93        43.483932     -80.567688    29011567      URBAN           GRASSLAND     DEFAULT_SOIL              [NONE]           [NONE]         2.068323      153.404199
  2901162002020 1  3.1907250000    369.81        43.479565     -80.613211    29011620      WETLAND         FOREST        DEFAULT_SOIL              [NONE]           [NONE]         3.262719      166.917507
  2901162003020 1  3.2528250000    369.81        43.470164     -80.617634    29011620      FOREST          FOREST        DEFAULT_SOIL              [NONE]           [NONE]         3.262719      166.917507
  2901162005040 1  1.7293500000    369.81        43.473755     -80.603662    29011620      URBAN           GRASSLAND     DEFAULT_SOIL              [NONE]           [NONE]         3.262719      166.917507
  2901162006030 1  10.4140125000   369.81        43.468539     -80.613626    29011620      AGRICULTURE     AGRICULTURE   DEFAULT_SOIL              [NONE]           [NONE]         3.262719      166.917507
  2901162006030 2  1.9843875000    369.81        43.493647     -80.628010    29011620      AGRICULTURE     AGRICULTURE   DRAINED_AGRICULTURE_SOIL  [NONE]           [NONE]         3.262719      166.917507
  2901280605040 1  2.8958933884    349.24        43.477448     -80.582251    29012806      URBAN           GRASSLAND     DEFAULT_SOIL              [NONE]           [NONE]         1.757881      160.431606
  2901280600000 0  0.6276066163    349.24        43.479611     -80.576567    29012806      LAKE            LAKE          LAKE                      [NONE]           [NONE]         1.757881      160.431606
  2901281005040 1  1.1505603664    341.40        43.472323     -80.561044    29012810      URBAN           GRASSLAND     DEFAULT_SOIL              [NONE]           [NONE]         1.747721      161.970201
  2901281000000 0  0.1697396368    341.40        43.473466     -80.556412    29012810      LAKE            LAKE          LAKE                      [NONE]           [NONE]         1.747721      161.970201
  2901663003020 1  1.0939500000    352.34        43.458178     -80.570019    29016630      FOREST          FOREST        DEFAULT_SOIL              [NONE]           [NONE]         2.289449      162.468970
  2901663005040 1  22.1620500000   352.34        43.459138     -80.548814    29016630      URBAN           GRASSLAND     DEFAULT_SOIL              [NONE]           [NONE]         2.289449      162.468970
  2901960502020 1  2.2119750000    376.80        43.482090     -80.645742    29019605      WETLAND         FOREST        DEFAULT_SOIL              [NONE]           [NONE]         3.490921      167.239563
  2901960506030 1  4.7117250000    376.80        43.476088     -80.643971    29019605      AGRICULTURE     AGRICULTURE   DEFAULT_SOIL              [NONE]           [NONE]         3.490921      167.239563
:EndHRUs

:RedirectToFile  Lakes.rvh
```

Figure 13. Inserting the edited HRU table into the RVH file. After reorganizing the HRU attributes in Excel, copy the HRU rows without the header row and paste them into the RVH file under the :HRUs block, replacing (or filling) the existing HRU entries while keeping the RVH block structure unchanged.

6. Discussions

Use the questions below to reflect on the HRU workflow and to connect GIS decisions with hydrologic modeling implications. Your answers should be supported by **calculations, figures (e.g., maps or plots), and citations** where appropriate.

(1) Can you derive **mean elevation, mean slope, and mean aspect** for each HRU using the **provincial DEM** provided in this walkthrough (instead of using subbasin-level attributes)? Compare the differences between subbasin-level and HRU-level results.

Use **Zonal Statistics as Table** tool. You are encouraged to consult **ArcGIS Pro documentation** of its usage.

(2) In this walkthrough, we generated preliminary HRUs using a **vector-based** Union of polygon feature classes. An alternative approach is to use **raster-based** overlay (e.g., converting land cover, vegetation, and soil to rasters and combining them). Discuss the **pros and cons of raster-based HRU generation** compared with the vector-based workflow. In your discussion, consider spatial resolution/cell size, boundary accuracy, computational efficiency, and how the choice may affect HRU simplification workflow.

You are encouraged to consult **public technical documentation** and **peer-reviewed journal articles** to support your discussion.

(3) In this lab we used an area threshold (e.g., 1 km^2) to remove sliver HRUs, but that **choice is not universal**. How do we know our **HRU simplification is "good enough"**?

- What **evidence** could you use to justify that 1 km^2 is reasonable for this basin and model purpose?
- What indicators would tell you the result is **too coarse** (over-simplified) or **too fine** (still too fragmented)?

Use information from your `HRU_raw` and `HRU_final` layers (e.g., HRU counts, area distributions, and maps) to support your answers.

(4) Eliminate is a geometry-based operation in ArcGIS and does not understand hydrologic meaning. What are the **limitations of using the ArcGIS "Eliminate" tool** to merge slivers?

- What kinds of unwanted changes can it introduce (e.g., merging across important boundaries, absorbing protected features)?
- How could you reduce these risks (e.g., selection rules, constraints like "merge only within SubId", protecting lakes/wetlands, post-merge QA checks)?

(5) HRUs translate GIS data (land cover, soil, vegetation) into model parameters and process representation. How can **HRU input data quality** affect hydrologic model results?

- What happens if land cover is outdated or static (e.g., a land cover map representing ~2010 conditions) while the simulation period spans many years?
- Which model components are most sensitive to these inputs (e.g., ET, snow accumulation/melt, infiltration, runoff generation), and how might errors show up in streamflow simulation?
- What improvements could be made in real projects (e.g., updated land cover, change detection, or scenario-based land cover inputs)?

(6) Do you think increasing the **number and diversity of HRUs** will necessarily improve model performance, or can overly fine HRU discretization sometimes reduce performance?

In your discussion, consider the resolution of forcing data, how parameters are assigned across HRUs, and how HRU complexity affects calibration uncertainty.

Appendix A: Building Semi-automated HRU Simplification Tool (HRU_simplify)

We use **ModelBuilder** in ArcGIS Pro (Analysis → ModelBuilder) to assemble a repeatable workflow for HRU simplification. ModelBuilder is a visual programming environment that allows you to chain multiple geoprocessing tools into a single model, manage intermediate outputs, and rerun the same procedure efficiently with different inputs. The model will (1) extract non-lake HRUs for a selected subbasin, (2) select sliver HRUs using an area threshold, and (3) eliminate slivers by merging them into adjacent larger HRUs. You will then repeat the model for other subbasins and, if needed, run a second round to remove residual slivers.

Go to Analysis → ModelBuilder (Figure A1), click it to create a model (default name is Model.tbx). Rename the Model (**Catalog** pane): In the **Catalog** pane, expand your toolbox, click once to select the model, then click again (slow double-click) on the model name to make it editable. Type the new name and press Enter. Use name **HRU_simplify**.

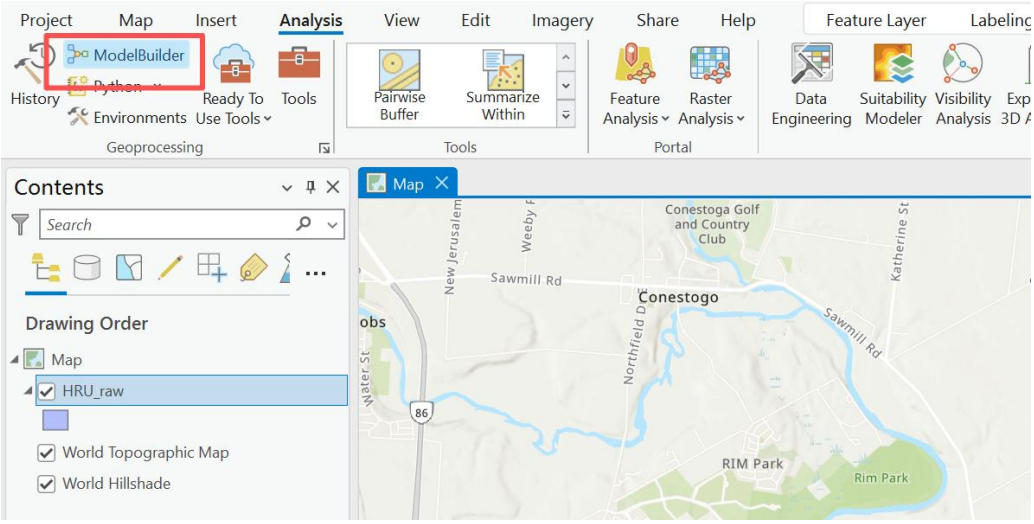


Figure A1. Example of opening **ModelBuilder** from the **Analysis** tab in ArcGIS Pro (highlighted).

Follow steps below to complete HRU_simplify model:

<u>Step 1: Add an iterator (Iterate Feature Classes)</u>

- From ModelBuilder, add **Iterate Feature Classes** (Iterators). See Figure A2.
- Set the **Workspace** to the geodatabase that contains the split outputs (e.g., MyProject8.gdb).
- Set a **Wildcard** to restrict the iterator to split inputs only, for example: **T*_0** (this matches all feature classes that start with T and end with _0, e.g., **based on names from Figure 8 Catalog pane**).
- The iterator provides two outputs: the current **Feature Class** and its **Name**. The Name variable is later used for automatic output naming.

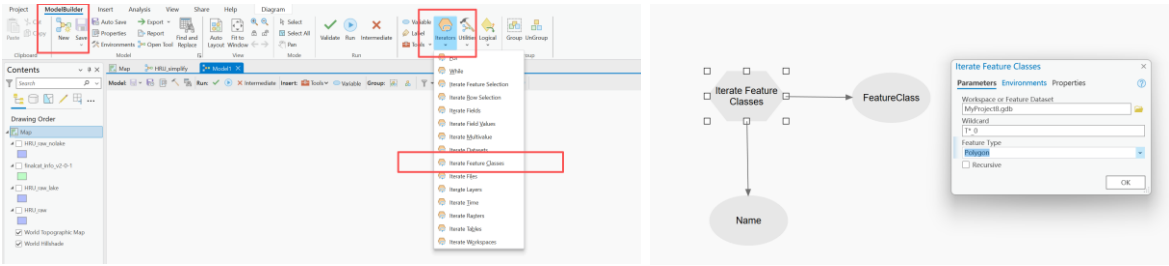


Figure A2. Adding an **Iterate Feature Classes** iterator in ModelBuilder (ModelBuilder tab → Iterators → Iterate Feature Classes) and configuring its workspace and wildcard (e.g., **T*_0**) to loop through the split subbasin feature classes.

Step 2: Convert the current feature class to a layer

Add **Make Feature Layer** and connect it to the iterator output feature class. This creates a temporary layer (e.g., output layer `HRU_subbasin_lyr`) that supports selection operations.

Step 3: Select slivers by area (Select Layer By Attribute)

Create a model parameter named **Sliver_threshold_km2** (Double) and set the default value to **1.0**. See Figure A3.

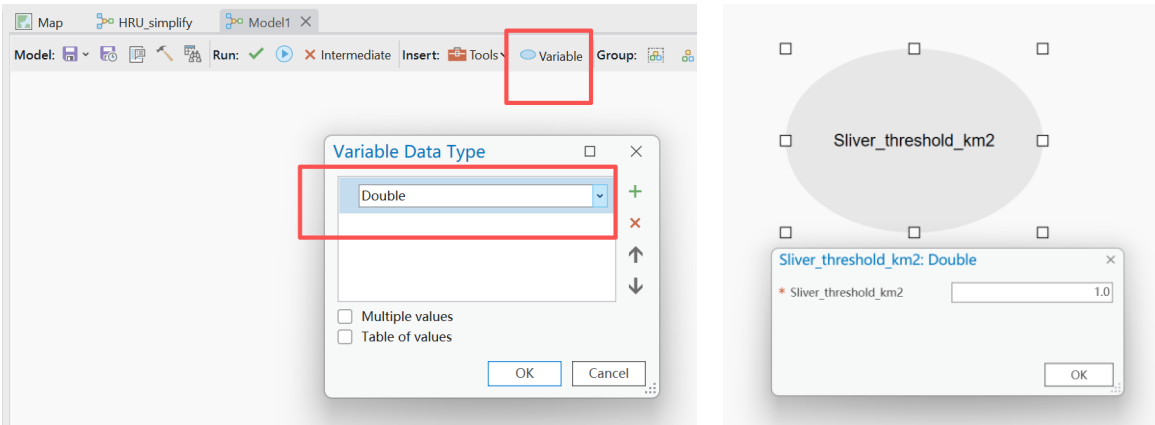


Figure A3. Creating a ModelBuilder **Double** variable to define the sliver area threshold (**Sliver_threshold_km2**), which is then used as an input parameter in the HRU simplification model.

Add **Select Layer By Attribute** and select slivers using an expression such as:

HRU_area < %Sliver_threshold_km2%

Note: In the ModelBuilder window, click **Tools** on the ModelBuilder toolbar to open the **Geoprocessing tool search**. Type **Select Layer By Attribute**, then **drag** the tool from the search results onto the model canvas.

Step 4: Eliminate slivers (Eliminate)

Add **Eliminate** (Data Management Tools → Generalization → Eliminate) and connect it to the selected layer. Eliminate merges the selected sliver polygons into adjacent polygons.

Step 5: Set unique output names using %BaseName%

When running multiple rounds (Run 1, Run 2, …), the iterator variable **%Name%** may already contain suffixes such as _run1, which can lead to stacked names (e.g., ..._run1_run2). To keep output names consistent (e.g., always **T*_0_runX**), add a **Calculate Value** step to create a "clean" base name.

In the **ModelBuilder** window, go to **Utilities → Calculate Value**, and drag it onto the model canvas. Open **Calculate Value** and set:

Data Type: String

Expression: clean_base(r"%Name%")

In the Code Block, define the **clean_base()** function to remove any existing run suffix:

```
import re

def clean_base(s):

    s = "" if s is None else str(s)

    return re.sub(r'_run\d+$', '', s)
```

Rename the output variable to **BaseName**. This BaseName variable is then used to construct output feature class names (e.g., %BaseName%_run1, %BaseName%_run2) without accumulating suffixes across rounds. Point BaseName to Eliminate.

Set the Eliminate output feature class name using inline variable substitution, for example:

%BaseName%_run1 for Run 1

This ensures each iteration writes a unique output (e.g., T29011620_0_run1, T29011620_0_run2, T29011620_0_run3) and prevents overwriting results from other subbasins.

Note: iterative runs and naming

The iterator model will generate one output per subbasin. If you need a **second** simplification round, you should avoid overwriting the first-round outputs by changing the output naming pattern. For example, set the Eliminate output to **%BaseName%_run1 for Run 1.** In second-round, reuse the same ModelBuilder workflow and change the **iterator input filter (Wildcard)** to **T*_0_run1** so the model processes the Run 1 outputs, and change the Eliminate output name to **%BaseName%_run2**. Repeat as needed until slivers are removed.

Step 6: Validate and run

Click **Validate** in ModelBuilder and fix any warnings (usually related to output paths). See Figure A4 for the full model workflow.

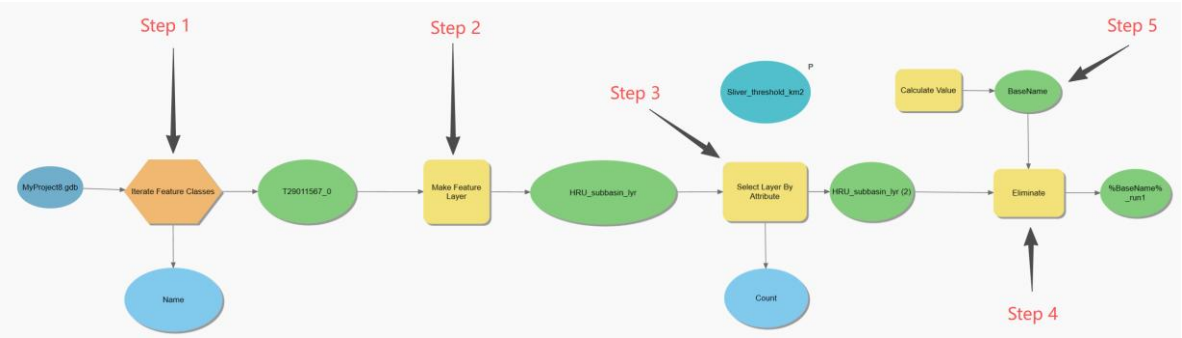


Figure A4. Full HRU simplification model workflow built in **ModelBuilder**. The four key steps are highlighted.

Save the model and run it once; it will iterate through all split subbasin feature classes automatically and generate one eliminated output per subbasin.